CS247 Assignment 1 Report

Slice Viewer for 3D Volume Data

The course material in this repository does not contain an explicit report template, only the grading rubric in `README.md` (minimum requirements 10+60+15+15 = 100 pts, plus optional bonus). This report is organised around that rubric: a short overview, a requirements coverage table, the implementation details that justify each item, screenshots and demo, and the macOS porting notes that were necessary to build the template on an Apple Silicon Mac.

1. Overview

The application loads an unsigned 16-bit CT volume into a single `GL_TEXTURE_3D`, then draws a full-screen quad whose fragment shader samples that texture. Switching the viewing axis or the slice index never re-uploads data. It only changes a single `mat4` uniform (`texTransform`) that controls how the quad's 2D parameter `(u, v)` maps to a 3D sample location. Aspect ratio is preserved at all window sizes by a second `mat4` (`model`) that scales the quad in NDC.

Two datasets are supported out of the box (`1` and `2` keys):

Dataset	Dimensions (x, y, z)
`lobster.dat`	120 × 120 × 34
`skewed_head.dat`	184 × 256 × 170

2. Requirements coverage

#	Requirement (from `README.md`)	Points	Where it is implemented
1	Download data into a 3D volume texture	10	`downloadVolumeAsTexture()` in `src/CS247_prog.cpp`. Sets `GL_TEXTURE_3D` filtering and wrap, then `glTexImage3D(..., GL_R16, ..., GL_RED, GL_UNSIGNED_SHORT, ...)`.
2	Display 3 different axis-aligned slices with GL texture mapping	60	`buildTexMatrix()` constructs a different `mat4` for `current_axis` ∈ `{0, 1, 2}`. The fragment shader does the actual 3D sampling via `texture(sampler3D, ...)`.
3	Adjustable slice position per axis	15	`W` / `S` keys mutate `current_slice[current_axis]`. The value is normalised to `[0, 1]` (voxel centre) when building `texTransform`. `A` cycles `current_axis`.
4	Correct data aspect ratio, no distortion on resize	15	`framebufferSizeCallback` updates `gFbWidth` / `gFbHeight` and `glViewport`. `buildModelMatrix()` then scales the quad by `min(sliceAspect/winAspect, winAspect/sliceAspect)` on the appropriate axis.
Total		**100**	

3. Implementation walk-through

3.1 Geometry: a unit quad in NDC

The renderable is a single quad in clip space, with its tex coords laid out as a unit square in the *xy* plane:

```
pos (-1,  1) — pos (1,  1)          tex (0, 1) — tex (1, 1)
   |              |                     |            |
pos (-1, -1) — pos (1, -1)          tex (0, 0) — tex (1, 0)
```

The geometry is uploaded once in `setup()` (VAO + VBO + EBO, 6 floats per vertex: position then tex coord, two triangles via EBO). Nothing about this VBO changes when the user switches axis or slice; everything is driven from uniforms.

3.2 The texture transform matrix

Each vertex carries a tex coord `(u, v, 0)`. The vertex shader applies `texTransform` and passes the result to the fragment shader as a 3D sampling location.

For each viewing axis the matrix is:

`current_axis`	Plane shown	`texTransform` action on `(u, v, 0, 1)`
0 (X)	Y–Z	`(sliceX/N_x, u, v, 1)`
1 (Y)	X–Z	`(u, sliceY/N_y, v, 1)`
2 (Z)	X–Y	`(u, v, sliceZ/N_z, 1)`

`sliceK + 0.5` is used in the numerator to sample at voxel centres. With `GL_LINEAR` filtering this gives clean bilinear interpolation inside the slice without re-uploading any data.

3.3 Aspect ratio correction

For each axis, the slice has its own pixel aspect:

Axis	Slice width × height
0	`dim_y × dim_z`
1	`dim_x × dim_z`
2	`dim_x × dim_y`

`buildModelMatrix()` compares `sliceAspect = sliceW / sliceH` to the framebuffer aspect `winAspect = fbW / fbH`, and shrinks whichever side is too wide so the slice fits without stretching:

```
if (winAspect > sliceAspect) sx = sliceAspect / winAspect;
else                         sy = winAspect   / sliceAspect;
```

The `glViewport` is updated in `framebufferSizeCallback` so this works correctly on retina displays too (framebuffer differs from window in points).

3.4 Shaders

`shaders/vertex.vs`:

```
#version 410 core
layout (location = 0) in vec3 pos;
layout (location = 1) in vec3 tCoord;
out vec3 texCoord;
uniform mat4 model;
uniform mat4 texTransform;
void main() {
    gl_Position = model * vec4(pos, 1.0);
    texCoord    = (texTransform * vec4(tCoord, 1.0)).xyz;
}
```

`shaders/fragment.fs`:

```
#version 410 core
in  vec3 texCoord;
uniform sampler3D volume;
out vec4 color;
void main() {
    float v = texture(volume, texCoord).r;
    color   = vec4(v, v, v, 1.0);
}
```

6. Controls

Key	Action
`1`	Load `data/lobster.dat`
`2`	Load `data/skewed_head.dat`
`A`	Cycle viewing axis (X → Y → Z → X ...)
`W`	Next slice along the current axis
`S`	Previous slice along the current axis
`B`	Cycle background clear color
`Esc`	Quit

A console message is printed on each axis or slice change for sanity.

7. Visual results

A short screen recording of the implementation in action is included next to this report as `demo.mov`. It shows loading both datasets, axis switching, slice scrubbing, and window resizing with aspect ratio preserved.